Smart Home Health Monitor

Final Presentation — CSC 494 IoT

Mausam Shrestha

Northern Kentucky University | Spring 2026

The Problem

Air purifiers lie about air quality.

Their built-in sensors are located **inside the device** — measuring air that has already been filtered.

> The air quality at the purifier = **always clean**
> The air quality where you breathe = **unknown**

Research shows:

- CO_2 above 1000 ppm causes measurable cognitive decline
- High PM2.5/NO_2 increases sleep issues by **50–60%**
- People spend **90%** of their time indoors

Why This Hasn't Been Solved

Issue	Commercial Purifiers	This Project
Sensor location	Inside device (cleaned air)	Breathing zone (3–6 ft)
Parameters monitored	PM2.5 only	CO2, PM2.5, VOC, Temp, Humidity
Works with any purifier	No — brand locked	Yes — via smart plug
Data export	Closed system	Open — Firebase + GitHub
Cost	$200–400 per room	**$88 total**

My Solution

A 5-sensor IoT monitor placed where you breathe

```
[5 Sensors at breathing height]
               |
     [XIAO ESP32-C6]
               |
   ────────────┼────────────────────────────
   |            |               |            |
[LCD]      [MQTT]        [Firebase]  [LED/
Display      Broker          Cloud     Smart Plug]
               |                 |
        [Purifier]       [React/PWA
                          Dashboard]
```

Hardware — What I Built

Component	Model	Protocol	Status
Microcontroller	XIAO ESP32-C6	—	✅ Deployed
CO2 Sensor	MH-Z19C (NDIR)	UART	✅ Working
PM2.5 Sensor	DC01	UART	✅ Working
VOC Sensor	MS1100	Analog	✅ Working
Temp/Humidity	AHT10	I2C	✅ Working
Display	LCD1602	I2C	✅ Working
Actuator	LED (smart plug stand-in)	GPIO	✅ Working

Total cost: $88 | All 5 sensors verified and deployed

Sprint 1 — Foundation (Weeks 1–6)

- ✅ GitHub repositories created and documented
- ✅ Canvas project + progress pages live throughout semester
- ✅ PPP presentation delivered (Week 4)
- ✅ Arduino IDE + XIAO ESP32-C6 configured
- ✅ Key discovery: XIAO uses **GPIO22/23** for I2C — not standard GPIO21/22
- ✅ AHT10, LCD1602, MH-Z19C, MS1100 individually tested + verified
- ✅ All 4 sensors running **simultaneously** on a single board

```
LCD Screen 1 (3 sec):     LCD Screen 2 (3 sec):
Temp: 24.1 C              CO2:  1600 ppm
Hum:  34.5%               VOC:  1874
```

Sprint 2 — Intelligence (Weeks 7–14)

Weeks 7–8: DC01 PM2.5 connected · MQTT broker live · LED automation triggered by thresholds · Firebase Firestore logging every 60 seconds

Weeks 9–10: React/PWA dashboard deployed · Real-time charts via Recharts · Firebase `onSnapshot` for instant updates · PWA installed on phone via manifest

Weeks 11–12: Sleep quality tracking interface added · 4+ weeks continuous data collected · Python ML model trained (scikit-learn)

Weeks 13–14: Moving average calibration applied · Documentation complete · Final presentation + demo video recorded

Automation Logic

The system automatically triggers the actuator based on sensor thresholds:

Condition	Action
CO_2 > 1000 ppm	LED **ON** (smart plug would activate purifier)
PM2.5 > 35 µg/m^3	LED **ON**
VOC spike detected	LED **ON**
All readings normal	LED **OFF**

Protocol: MQTT over Wi-Fi — latency < 2 seconds

> **Demo**: LED used as actuator stand-in. In real deployment, a Tasmota-flashed smart plug replaces the LED — same MQTT message, same automation logic, any purifier brand.

Cloud Dashboard — React + PWA

- Real-time sensor readings — Firebase `onSnapshot` pushes every update instantly
- Historical charts for CO_2, PM2.5, VOC, Temp, Humidity (24h / 7d / 30d)
- Threshold alert indicators on dashboard
- Sleep quality rating interface — logs nightly scores to Firebase
- **PWA**: add to phone home screen from Safari or Chrome — no App Store needed
- Works on iOS and Android — single React codebase

Demonstration

🎬 Demo Video

What the video shows:

1. Sensors reading live — CO2 rising in a closed room
2. CO2 threshold exceeded → LED turns on automatically
3. Firebase dashboard updates in real time
4. Historical charts and 30-day trend view
5. PWA installed on phone — launching from home screen
6. Sleep quality log entry

Demo video: https://drive.google.com/drive/folders/1dxNxLwAFDM-qzTNDedeU8n3i4qP3a_3x?usp=sharing

Data Collected — 4+ Weeks Continuous

Bedroom deployment results:

Metric	Typical Range	Notable Events
CO_2	450–1800 ppm	12 nights exceeded 1000 ppm
PM2.5	2–28 µg/m^3	4 events exceeded 35 µg/m^3
VOC	100–600	Morning spikes (cooking)
Temp	18–24°C	Stable throughout
Humidity	30–55%	Lower in winter months

> **Key finding**: CO_2 consistently exceeded 1000 ppm between 11pm–6am in a closed bedroom — correlating directly with lower sleep quality scores.

Machine Learning Results

Model: Random Forest Regressor — Python + scikit-learn

Target: Next-night sleep quality score (1–10 scale)

Features: Nightly avg CO_2, PM2.5, VOC, temperature, humidity

Metric	Value
Training nights	35
R^2 score	0.74
MAE	0.8 points
Top predictor	Nightly average CO_2

Insight: Nights with CO_2 below 800 ppm → avg sleep score 7.4/10.

Nights above 1200 ppm → avg sleep score 5.2/10.

Technology Stack

Hardware/Firmware

- XIAO ESP32-C6 · Arduino IDE · C++
- Libraries: Adafruit AHTX0, LiquidCrystal_I2C, MHZ19
- Protocols: I2C, UART, Analog, MQTT

Backend / Cloud

- Firebase Firestore — real-time time-series data storage
- MQTT broker — actuator control, < 2s latency

Frontend / Mobile

- React + PWA — single codebase for desktop and mobile
- Recharts — real-time sensor data visualization

Learning with AI

Topic 1: Multi-Sensor Integration & Calibration

Used AI to understand XIAO-specific pin mapping, debug I2C bus sharing, implement UART protocol for MH-Z19C, and apply moving average calibration

Topic 2: Mobile App Development with React/PWA

Used AI to scaffold React dashboard components, implement Firebase real-time listeners, structure Recharts time-series visualization, and build PWA manifest

> **Primary AI tool**: Claude (Anthropic) — used for concept explanations, code debugging, protocol understanding, and documentation throughout both sprints

What I Would Do Differently

1. **Start data collection in Week 4** — more than 4 weeks would strengthen the ML model significantly
2. **Swap DC01 for PMS5003** — PMS5003 uses laser scattering for PM2.5; more accurate than DC01's infrared measurement
3. **Real Tasmota smart plug** — The LED demo proves automation logic; a smart plug adds real appliance control with zero code changes
4. **Multi-room support** — A second XIAO unit would allow comparing bedroom vs. living room air quality in real time

Results Summary

- ✅ **Problem solved**: Sensors at breathing zone, not inside purifier
- ✅ **5 sensors running 24/7** for 4+ weeks uninterrupted
- ✅ **Automated actuator control** triggered by CO_2 + PM2.5 thresholds via MQTT
- ✅ **Public PWA dashboard** — real-time + 30-day historical data, installable on any phone
- ✅ **ML model trained** — $R^2 = 0.74$, predicting sleep quality from air sensor data
- ✅ **Total cost: $88** vs. $200–400 commercial alternatives
- ✅ **Open source** — all code and data on GitHub

Repository Links

Project Code & Documentation

https://github.com/shresthamausam07/smart-home-health-monitor

Learning with AI Documentation

https://github.com/shresthamausam07/mausam-shrestha-learning-with-ai

Canvas Project Page

https://nku.instructure.com/courses/88152/pages/individual-project-mausam-shrestha

Canvas Progress Page

https://nku.instructure.com/courses/88152/pages/individual-progress-mausam-shrestha

Live Dashboard

https://smart-home-health-monitor.web.app

Thank You

Any Questions?

Mausam Shrestha

shrestham2@mymail.nku.edu

CSC 494 — IoT | Spring 2026

Northern Kentucky University

> *"The goal was to know what you're actually breathing — not what the purifier thinks you're breathing."*